

IMPERIAL

MENG INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Tools for Foundational Verification of Probabilistic Programs

Author:
Edwin Fernando

Supervisor:
Shing Hin Ho
Dr. Azalea Raad

Second Marker:
Prof. Nicolas Wu

June 1, 2026

Contents

1	Overview	3
1.1	Contributions	3
1.2	Motivation	4
1.2.1	Discrete vs Continuous distributions	4
1.2.2	Probabilistic programs in the wild	4
1.3	Lean and related formalisation projects	4
2	Background	6
2.0.1	Monads	6
2.0.2	Type Theory	6
2.1	Measure Theory	6
2.1.1	Borel spaces and lebesgue measures	7
2.1.2	Product of measurable spaces	7
2.1.3	Probability THoery: (independent) random variables, joints and marginals, conditioning, expection	7
2.2	Semantics of Probabilistic Programs	7
2.2.1	Types in BPPL	8
2.2.2	The let binding problem	10
2.2.3	Semantics of Let binding	10
2.3	Separation Logic	11
2.3.1	Classical separation logic	12
2.3.2	Zoo of separation logics	12
2.3.3	Iris and MoSeL	15
2.4	Theorem Proving	15
2.4.1	Dependent type theory	15
2.4.2	Metaprogramming	18
2.5	Instantiating Iris	19
2.5.1	Deep vs Shallow Embeddings	19
2.5.2	BI in Iris	20
2.5.3	First Attempt: Deeply embedded logic instantiation	20
2.5.4	Shallow embedding of the logic	21
2.6	Deparametrisation of LProp	21
2.6.1	Substitution inconsistency	22

Chapter 1

Overview

1.1 Contributions

- We provide the first complete implementation of a probabilistic separation logic. Specifically Lilac is formalised in the Lean theorem prover complete with both soundness proof and extensive support for tactic-based weakest-precondition style reasoning about programs
- One of the first non-trivial instantiation of the Lean port of the Iris framework (originally developed in Rocq).
- A Novel treatment of infinite looping programs which may construct a convergent sequence of values. The correctness of some algorithms for probabilistic inference is given by convergence of such a sequence. So add a connective to Lilac for reasoning about infinite loops and re-establish the soundness of the logic.

“The search for well-specified probabilistic models—models that correctly describe the processes that produce data—is one of the enduring ideals of the statistical sciences. Yet, in only the simplest of settings are we able to achieve this goal” - Papamakarios et al

“Although our tactics perform symbolic execution in small steps, one can easily define a tactic that performs symbolic execution repeatedly. However, in concurrent separation logic, small step symbolic execution is often needed so as to be able to open invariants around atomic expressions” - Krebbers et al. 2017b <https://iris-project.org/pdfs/2017-popl-proofmode-final.pdf>

so it makes sense for be to automatically apply `wp_let`, `wp_ret` and `wp_unif` and `wp_ber` immediately. However is it not possible for this to get stuck if we are not dealing with `unif` and `ber` alone (eg. `complicate (det/rand)val` coming from the meta logic??)

Shall I talk about logic programming via type classes? Probably briefly should do. To illustrate why and how certain type classes were implemented for certain propositions. Especially interesting is this line [Krebbers et al. 2017b] “However, since we use a shallow embedding to represent the propositions of our object logic, propositions are semantic objects, so we cannot just write a Coq function to perform this traversal. Instead, we use logic programming using type classes to perform such manipulations on the meta level”

This work is a good example exhibiting an easy to use separation logic proofmode, without the complexity of step-indexing (as in most iris projects). This is because our language is first-order and does not support recursion and yet has an orthogonal axis of intricacy which makes it interesting. As such it makes a good case study for the application of modern proofmode construction techniques decoupled from the step-indexed world of Iris in which practically every other significant program logic lives.

Expressive tactic-based reasoning about programs is developed through Lean meta-programming.

1.2 Motivation

Here we give an informal account of what probabilistic programs are and why we would care to use them A Probabilistic programming language is not an established or general paradigm of programming such as Functional or Imperative programming. Most generally probabilistic programs are simply programs which have a `sample` construct for sampling from probability distributions, and optionally, an `observe` for specifying a new distribution given an old distribution and some newly observed data.

Statisticians use languages such as Stan [Carpenter et al. \[2017\]](#), Hakaru [Narayanan et al. \[2016\]](#) and Anglican [Wood et al. \[2014\]](#), to build complex statistical models succinctly using the expressivity of a programming language. Note that here, a model is simply a probability distribution over some quantity of interest, take for instance the probability of a 3d structure being correct for the given protein sequence, (other examples may come from weather forecasting, medical diagnosis, economics, statistical mechanics etc). "Running" this code corresponds to running a simulation of the model, where we can get answers such as the mean or variance, or just samples (sample protein structure to test in a lab in the example).

What distinguishes a probabilistic programming language is that it decouples the specification of a statistical model from the algorithm for its simulation (henceforth called probabilistic inference). Inference algorithms are often highly optimised and specialised to the specific model and the interpreter of a PPLs attempts to make the best choice/combination of inference algorithms for the given code (model), relieving the user of this additional work.

So in other words PPLs are a layer of abstraction which sits between a statistical model and the inference algorithms which simulate it. This abstraction unlocks the verification of correctness properties of the statistical model, which led to the development of program logics for reasoning about PPLs. This will be the main topic of this report.

1.2.1 Discrete vs Continuous distributions

First let us define what discrete and general (continuous) PPLs are. A discrete PPL does not allow specifying a probability distribution over $\mathbb{R}$. This is achieved most straightforwardly by not including a type corresponding to $\mathbb{R}$ in the syntax of a PPL. A general PPL will have a type corresponding to $\mathbb{R}$. This seemingly small detail has significant consequences on the semantics of a PPL which in turn dictates the complexity of program logics for such a language. This dichotomy largely exists due to difficulties with semantics of continuous PPLs. Take for example that a semantics for higher order PPLs was given by [\[Staton, 2017\]](#) within the decade.

However, all the PPLs above are continuous. Indeed PPLs are made only relevant when working with continuous distributions, as they remove the difficulty of working with the probabilistic inference algorithms. Much less sophistication is required in the inference algorithms for discrete algorithms, so discrete PPLs are not of much interest. So why are program logics for discrete PPLs such as [\[Bao et al., 2025\]](#) and [Lohse et al. \[2026\]](#) important?

1.2.2 Probabilistic programs in the wild

In the most general sense a probabilistic program is just any program which samples from a distribution. Support for this can be found in the standard libraries of every major programming language, such as Python, C++ or Java. Some of the critical applications manipulating discrete distributions, where formal verification is desirable is Cryptography and differential privacy. In fact one of the first major applications of probabilistic *separation* logic is being pursued in the formalisation of [\[Bao et al., 2025\]](#) to prove soundness of Zero Knowledge Proof Systems (subfield of Cryptography).

1.3 Lean and related formalisation projects

Foundational verification, consists of machine checked proofs of theorems, assuming only the axioms of mathematics. Here we are concerned with verification of software, so the theorems in question express properties of the software. The axioms of mathematics that we use is a dependent type

theory. In this setting of dependent type theory, checking the correctness of proof is the same thing as checking if a proof (program) is well typed, due to the Curry-Howard Isomorphism. Further the tool we use to for machine-checked proofs is an interactive theorem prover with support for proofs using "tactics". Tactical support is just metaprogramming, trying to encapsulate reasoning principles a person would use when writing pen-and-paper proofs. There are two proof assistants that were strongly considered for the purposes of this project: Lean and Rocq, former having proofs of required theorems in measure theory and the latter having the original implementation of the Iris framework. Lean was chosen since the front-end of the Iris proof mode (MoSeL, see [2.3.3](#)) had been fully ported to Lean very recently.

There has been an exciting amount of recent activity in formalisations related to this work, which has enabled the formalisation of Lilac (see, ??) in Lean to be a realistic goal. Most importantly, the formalisation of the disintegration theorem in [Degenne \[2025\]](#) is only available in Lean at the moment. Disintegration theorem also involved the formalisation of kernels which are actually available in several other proof assistants. Quasi-borel spaces [Heunen et al. \[2017\]](#) (in which higher order probabilistic programs are interpreted) have been formalised in Isabelle/HOL in [Hirata et al. \[2023\]](#), along with kernels. s-finite-kernels can also be found in rocq [Affeldt et al. \[2025\]](#).

A lot of background in Lean was required to undertake this project. For example, we use the full expressivity of inductive types and inductive families. Also an understanding of propositional vs definitional equality which is related to Proof irrelevance [Avigad et al. \[2024\]](#) and how that's implemented in Lean's type theory by special treatment of the lowest Universe [Carneiro \[2019\]](#), is very helpful.

Chapter 2

Background

Probabilistic programming is a paradigm where we are defining statistical models using code. It is a means of describing complex distributions over some space in a structured way. Probabilistic programming language (PPL) also come with an interpreter (probabilistic inference engine) which allows the programmer to sample a term from the distribution denoted by a program and indeed sample distributions while constructing new distributions.

More interestingly a Bayesian probabilistic programming languages (BPPL) also allows one to "observe" an event and update their belief of the world (update the probability distribution they associate with some occurrence). Here we take the bayesian perspective underpinned by Baye's law which describes how we update a probability distribution based

A simple example taken from [Staton \[2017\]](#) is reproduced below to show how

2.0.1 Monads

The reader is assumed to have had been exposed to a monadic style of programming (for example Haskell). Mathematical/categorical perspective is not strictly necessary, and is thus omitted.

Here we give a brief refresher on the computational perspective of monads It is relevant to understand the semantics of probabilistic programming (Giry Monad) and Leans metaprogramming library (MetaM monad).

Nevertheless the categorical/mathematical perspective on monads though not strictly necessary to understand this work, it's a valuable alternative perspective. For example the justification for the semantics of higher order probabilistic programs being an open problem till 2017, if given naturally in categorical language (the category of measurable spaces in which types are interpreted is not Cartesian closed).

From the computational perspective, A monad can be understood as a map from type A to $M A$. It is additionally associated with two function `Ret` and `bind`, with the following your signatures. Monads are actually monoids with `Ret` acting as unit and `bind` acting as the binary operation. The laws that `Ret` and `bind` obey are as follows which should remind one of the monoid laws.

2.0.2 Type Theory

Readers are also assumed to have some familiarity with type theory, in particular having seen inference rules.

2.1 Measure Theory

In order to understand the semantics of a probabilistic programming, a formal definition for probability is needed. Measure theory underpins probability theory and an overview is given here. We want to be able to talk about the probabilities of some event occurring, which is modelled by a random variable. In order to define a random variable we first need to define Measures and

Measurable Space. A measurable space is a pair (Ω, Σ_Ω) , where Ω is some Set, and $\Sigma_\Omega \subseteq \mathcal{P}(\Omega)$ is called the σ -algebra. We interpret each element of Σ_Ω to be an event (such as a coin flip landing heads). We would like the σ -algebra to be closed under complements and countable unions. From the presentation so far, it may seem like all singleton sets must be events but due to technical reasons (see [Axler \[2020\]](#) Ch. 2), this cannot be in certain cases where Σ is infinite.

A measure on a measurable space is a function $\mu : \Sigma_\Omega \rightarrow [0, \infty]$.

A probability measure on the other hand is more restrictive: $\mu : \Sigma_\Omega \rightarrow [0, 1]$. It gives the probability of an event $E \in \Sigma_\Omega$ occurring. A similar but more general notion than what a measure gives us, is a random variable: a measurable function between two measurable spaces. I say "more general notion", since using the identity measurable function yields the measure we started with. A measurable function denoted $f : (\Omega_1, \Sigma_{\Omega_1}) \xrightarrow{m} (\Omega_2, \mathcal{G})$ $f : \Omega_1 \xrightarrow{m} \Omega_2$ is well defined when there is an ambient measurable space structure $(\Omega_1, \Sigma_{\Omega_1})$ and $(\Omega_2, \mathcal{G})$. f allows us to talk about the distribution of a particular attribute of in the event space. For example $\Omega_1 = \{1, \dots, 6\}^2$ may be the set of all events associated with playing 2 dice, while f may map to the sum of the two dice, $\Omega_2 = \{1, \dots, 12\}$. A measurable function f , transforms any measure μ on $(\Omega_1, \Sigma_{\Omega_1})$ to the pushforward measure μ' on $(\Omega_2, \mathcal{G})$. In order for this to always be well defined we require the following measurability to be satisfied by all measurable functions: $\forall G \in \mathcal{G}, f^{-1}(G) \in \Sigma_\Omega$.

A Lebesgue integral is a generalisation of the standard (Reimann) integral. Intuitively the lebesgue and reimann integral will coincide on the spaces such as $\mathbb{R}$, where the domain of integration is continuous. The lebesgue integral relaxes that requirement. The lebesgue integral of a measurable function $f : \Omega \rightarrow [0, \infty]$ with respect to a measure $\mu : \Sigma_\Omega \rightarrow [0, \infty]$ is a number defined by:

$$\int_\Omega f d\mu := \sup \left\{ \sum_{i=1}^n f(\omega_i) \mu(U_i) \mid \{U_i \in \Sigma_\Omega\}_{i=1}^n \text{ disjoint cover of } \Omega \right\}.$$

To get an intuition remember that integrals are supposed to add the values of function by breaking the domain up into infinitesimal pieces. Observe that the integral is a weighted sum of $f(\omega)$, weighted by the measure. The integral is iterating over the entire space by picking some choice of measurable disjoint sets whose union is the Ω (disjoint cover of Ω). And for each measurable set (U_i) it picks a representative point ω evaluates f on ω and weights it by how large the set is (μU_i). The supremum and infimum together enforce that the chosen disjoint cover is chosen as finely (small measure) as possible.

2.1.1 Borel spaces and lebesgue measures

2.1.2 Product of measurable spaces

2.1.3 Probability Theory: (independent) random variables, joints and marginals, conditioning, expectation

?? In the definition of E and M, we want them to be as general as possible, so we take the measurable space which is very fine grained. In other words it is very large as it contains a large number of sets. To bolster the intuition that the measurability condition is less restrictive over finer measurable space in the domain, Imagine a coarse measurable space in the domain. The pre-image of the function is less likely to be a measurable set in the domain, and satisfy the measurability condition. Hence coarser measurable spaces in the domain are more restrictive. It is helpful to think of the measurable space in the domain of RVA as the finest (greatest) measurable space (though this doesn't exist for general Ω), until we choose a specific Ω and measurable space on it later on.

2.2 Semantics of Probabilistic Programs

We first present APPL [[Li et al., 2023](#), appendix A], a probabilistic programming language without bayesian updates. Syntactically it looks like a basic WHILE language. In Appl, there isn't an explicit sample construct, sampling is realised in the semantics of sequencing as explained later. Though the language is first order, PPLs still have a non-trivial, novel denotational semantics. The

central difference is that types are interpreted as `MeasurableSpace` rather than just `Set` from which everything else follows. Concretely $\llbracket A \rrbracket = (\mathbb{A}, \Sigma_{\mathbb{A}})$. In our PPL we would like to express measures and manipulate measures. The type constructor `GA` is interpreted as the (meta) type $\text{ProbMeasure}[\llbracket A \rrbracket]$, the `Set MeasurableSpace` of measures over $\llbracket A \rrbracket$. So in order for `GA` to be a valid (object) type, there must be some canonical `MeasurableSpace` $\Sigma_{GA} \subseteq \mathcal{P}(\Sigma_A \rightarrow [0, 1])$. This is indeed true, but we don't reprove the mathematical foundations on which the semantics stands, in this report.

<insert: Why measurableSpaces?>

$$\begin{aligned}
A, B &::= A \times B \mid \text{bool} \mid \text{real} \mid A^n \mid \text{index} \mid \text{GA} \\
p &\in [0, 1] \\
r &\in \mathbb{R} \\
n &\in \mathbb{N} \\
\oplus &\in \text{Arith} := \{+, -, \times, /\} \\
< &\in \text{Cmp} := \{<, \leq, =\} \\
M, N, O &::= X \mid \text{ret } M \mid X \leftarrow M; N \mid \\
&\quad (M, N) \mid \text{fst } M \mid \text{snd } M \mid \\
&\quad T \mid F \mid \text{if } M \text{ then } N \text{ else } O \mid \text{flip } p \mid \\
&\quad r \mid M \oplus N \mid M < N \mid \text{unif } [0, 1] \mid \\
&\quad [M, \dots, M] \mid [M|N] \mid \text{for}(n, M_{\text{init}}, i \ X. M_{\text{step}})
\end{aligned}$$

Figure 2.1: Syntax of the language

$$\begin{aligned}
\llbracket A \times B \rrbracket &= \llbracket A \rrbracket \otimes \llbracket B \rrbracket \\
\llbracket \text{bool} \rrbracket &= (\{T, F\}, \mathcal{P}(\{T, F\})) \\
\llbracket \text{real} \rrbracket &= (\mathbb{R}, \mathcal{B}(\mathbb{R})) \\
\llbracket A^n \rrbracket &= \underbrace{\llbracket A \rrbracket \otimes \dots \otimes \llbracket A \rrbracket}_{n \text{ times}} \\
\llbracket \text{index} \rrbracket &= (\mathbb{N}, \mathcal{P}(\mathbb{N})) \\
\llbracket \text{GA} \rrbracket &= \mathcal{G}[\llbracket A \rrbracket] \\
\llbracket \mathbf{1} \rrbracket &= \text{the one-point space}
\end{aligned}$$

Figure 2.2: Denotational semantics of types

2.2.1 Types in BPPL

The setting is the category of Measurable Spaces, `Meas`. All types in our language is interpreted as an object in this category. Syntactically:

$$\mathbb{A}, \mathbb{B} ::= \mathbb{R} \mid \mathcal{P}(\mathbb{A}) \mid 1 \mid \mathbb{A} \times \mathbb{B} \mid \dots$$

Objects in this category are of the form (A, Σ_A) . Morphisms are measurable functions. The interesting type here is $\mathcal{P}(\mathbb{A})$. It's the type of probability distributions over $\mathbb{A}$. Actually not quite, since we allow any measure $\mu : \Sigma_A \rightarrow [0, \infty]$, rather than $\Sigma_A \rightarrow [0, 1]$. Now in order to be an object in the category, $\mathcal{P}(\mathbb{A})$, the set of all measures on $\mathbb{A}$, must itself form a measurable space. There is a construction to form a σ -algebra for $\mathcal{P}(\mathbb{A})$ satisfying all the requirements (we don't go into it here).

A natural next question is how these measures can be created and manipulated. The semantics of PPLs are given in a monadic framework (in other words probabilistic programs can be thought of as effectful computations). In practice this means that there is `ret` and `bind` type constructors to construct terms of (object) type $\text{mathrm{G}}?$, such that

$$\begin{aligned}
\llbracket X \rrbracket \rho &= \rho(X) \\
\llbracket \text{ret } M \rrbracket \rho &= \text{ret } \llbracket M \rrbracket \rho \\
\llbracket X \leftarrow M; N \rrbracket \rho &= v \leftarrow \llbracket M \rrbracket \rho; \llbracket N \rrbracket [X \mapsto v] \\
\llbracket (M, N) \rrbracket \rho &= (\llbracket M \rrbracket \rho, \llbracket N \rrbracket \rho) \\
\llbracket \text{fst } M \rrbracket \rho &= \pi_1(\llbracket M \rrbracket \rho) \\
\llbracket \text{snd } M \rrbracket \rho &= \pi_2(\llbracket M \rrbracket \rho) \\
\llbracket \text{T} \rrbracket \rho &= \text{T} \\
\llbracket \text{F} \rrbracket \rho &= \text{F} \\
\llbracket \text{if } M \text{ then } N \text{ else } O \rrbracket \rho &= \begin{cases} \llbracket N \rrbracket \rho, & \llbracket M \rrbracket \rho = \text{T} \\ \llbracket O \rrbracket \rho, & \llbracket M \rrbracket \rho = \text{F} \end{cases} \\
\llbracket \text{flip } p \rrbracket \rho &= \text{Ber } p \\
\llbracket r \rrbracket \rho &= r \\
\llbracket M \oplus N \rrbracket \rho &= (\llbracket M \rrbracket \rho) \oplus (\llbracket N \rrbracket \rho) \\
\llbracket \text{unif } [0, 1] \rrbracket \rho &= \text{Unif } [0, 1] \\
\llbracket [M_1, \dots, M_n] \rrbracket \rho &= (\llbracket M_1 \rrbracket \rho, \dots, \llbracket M_n \rrbracket \rho) \\
\llbracket [M[N] : A] \rrbracket \rho &= \begin{cases} \pi_N(\llbracket M \rrbracket \rho), & 1 \leq \llbracket N \rrbracket \rho \leq n \\ \text{arbitrary}(A), & \text{otherwise} \end{cases} \\
\llbracket \text{for}(n, M_i, i \leftarrow X. M_s) \rrbracket \rho &= \text{loop}(1, \llbracket M_i \rrbracket \rho, \lambda k. \llbracket M_s \rrbracket \rho[i \mapsto k, X \mapsto v]) \\
\text{where } \text{loop}(k, u, f) &= \begin{cases} \text{ret } u, & k > n \\ v' \leftarrow f(k, v); \text{loop}(k+1, v', f), & \text{otherwise} \end{cases}
\end{aligned}$$

Figure 2.3: Denotational semantics of terms

$$\llbracket \text{ret} \rrbracket : A \rightarrow \mathcal{G}A \quad \llbracket \text{bind} \rrbracket : \mathcal{G}A \rightarrow (A \rightarrow \mathcal{G}B) \rightarrow \mathcal{G}B$$

The core intuition of how probabilistic programs work is gained by studying the semantics of let bindings `let`. But first let's explain how variables work in Appl. A standard technique for representing variables is to use a variable context. A map from this context to the denotation of an AST would be considered a program. However, in Appl, the context is a measurable space, and the map is also measurable. Precisely the context and interpretation of well-typedness in context is given as:

$$\llbracket \Gamma, X : A \rrbracket = \llbracket \Gamma \rrbracket \otimes \llbracket A \rrbracket \quad \llbracket M \rrbracket : \llbracket \Gamma \rrbracket \xrightarrow{m} \llbracket A \rrbracket$$

2.2.2 The let binding problem

Staton provided a compositional semantics for first-order Bayesian probabilistic programs in [Staton \[2017\]](#), which forms the foundation of program logics, validation of inference algorithms, etc. The scope of this project is entirely within first-order probabilistic programs (probability distributions over function spaces are not expressible). The programming language considered in Lilac uses a simpler semantics since it doesn't support bayesian updates. So in the following, we present the relevant excerpt of the semantics.

The key contribution of the paper is the introduction of `s-finite kernels`, motivated by validating a basic commutativity property that programmers expect. Specifically: when neither `x`

$$\begin{array}{ccc} \text{let } x = t \text{ in} & & \text{let } y = u \text{ in} \\ \text{let } y = u \text{ in} & = & \text{let } x = t \text{ in} \\ v & & v \end{array}$$

Figure 2.4: commutative program transformation

is free in `u` nor `y` is free in `t`. Another benefit of this semantics is a term-wise compositionality, ie, the semantics of every subterm in a program is well-defined, not just whole programs. This is just another way of saying the semantics of open programs (those with free variables), as well as closed programs is well-defined, respectively. Validating proof rules in a logic is much better formulated in terms of such a compositional semantics. `s-finite kernels` denote open programs and `s-finite measures` denote closed programs.

At this point it is important to disseminate the difference between Lilac's lack of bayesian updates (call this APPL) and the more general language with bayesian updates (call this BPPL). As justified in the `??`, it is necessary to work with (unnormalised) measures when

2.2.3 Semantics of Let binding

`??` Probabilistic programs are interpreted using the Giry monad [Giry \[1982\]](#) in the category of Measurable Space, `Meas`. Probability is an effect. This interpretation gives us the semantics of sequencing (let-binding) so we go through it below. Kernels also naturally show up in type of the monadic `bind`, which motivates it.

The Giry functor takes some object (measurable space) (A, Σ_A) to a measurable space on the measures of $(\Sigma_A \rightarrow [0, \infty], \dots)$ (the σ -algebra is omitted). We'll denote $\Sigma_A \rightarrow [0, \infty]$ as `Measure A`, for readability.

The explanation focuses on intuition through diagrams. For a measurable space (A, Σ_A) , a probability measure is of type $\mu : \Sigma_A \rightarrow [0, 1]$. This gives a distribution on `A`. The intuition behind μ is that we can sample some event using it. A kernel from `A` to `B` has type $\kappa : A \rightarrow \text{Measure } B$. Each triangle below is a kernel. And the types of it input and output are indicated, and we compose the kernels along the type `B` as $\kappa \circ_B \iota$. The types of the individual kernels and composed kernel of Fig `??`, is given below

$$\iota : A \rightarrow \text{Measure } B \quad (2.1)$$

$$\kappa : B \rightarrow \text{Measure } C \quad (2.2)$$

$$\kappa \circ_B \iota : A \rightarrow \text{Measure } C \quad (2.3)$$

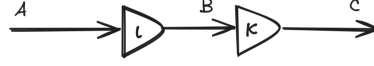


Figure 2.5: A simple probabilistic program

The interpretation of the kernel ι is that for every (deterministic) value of A it gets, it will give you a (probabilistic value) distribution over type B . So composition here, is sampling from the Measure B , getting a deterministic value of type B and passing that to the kernel κ . This sampling is done ranging over all the values of B in proportion to their distribution, and taking a weighted average of the values of type Measure C . More precisely, this is the integral: $\kappa \circ_B \iota = \lambda a. \int \lambda b. \kappa(b) d\iota(a)$ (integral will be explained in 2.1 in the final draft). Notice that $\circ$ has a type very similar to the monadic bind (in the Girmonad). We now write $\mathcal{G}(A)$ instead of Measure A , for the Girmonad functor. $\circ : ((B) \rightarrow \mathcal{G}(C)) \rightarrow ((A) \rightarrow \mathcal{G}(B)) \rightarrow \mathcal{G}(C)$. If we just swap the arguments and partially apply an $a : A$ on ι , we see that the bind of the Girmonad composes a measure with a kernel like so $\iota(a) \gg \kappa : \mathcal{G}(B) \rightarrow (B \rightarrow \mathcal{G}(C)) \rightarrow \mathcal{G}(C)$

The most interesting part of Staton’s commutative semantics is that when kernels have multiple inputs (this does extend into the realm of multi-categories but we don’t go into that here). To clarify, these kernels have multiple deterministic values they take in order to return a single probabilistic value (a measure). The commutativity property illustrated by the equality of the two programs in Fig ??.

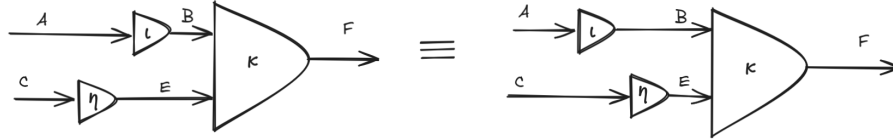


Figure 2.6: Commutativity of let bindings

$\kappa \circ_B \iota$ is given by the following on the left and right respectively

$$\lambda a c. \int \lambda e. \left(\int \lambda b. \kappa(b, e) d\iota(a) \right) d\eta(c)$$

$$\lambda a c. \int \lambda b. \left(\int \lambda e. \kappa(b, e) d\eta(c) \right) d\iota(a)$$

Though these look rather unreadable, all that’s happening is swapping the order of integration, and there is a theorem which allows us to this in certain conditions. In Staton [2017] it is shown that this theorem holds if instead of general kernels, we restrict ourselves to s-finite-kernels. A kernel $\kappa : A \rightarrow \mathcal{G}(B)$ is finite if there is finite $r \in [0, \infty)$ such that, for all a , $\kappa(a, B) < r$. κ is s-finite if there is a sequence $\kappa_1 \dots \kappa_n \dots$ of finite kernels and $\sum_{i=1 \dots \infty} \kappa_i = \kappa$.

In short topological transformations of these diagrams corresponding to the motivating example in 2.4 do not change the semantics.

2.3 Separation Logic

Separation logic has been extremely productive in the last two decades in reasoning about imperative programming languages, after first being introduced in O’Hearn et al. [2001]. The classical separation logic introduced a compositional state, specifically a heap (modelled as a function

$\mathbb{N} \rightarrow \text{Val}$), which was required along with a more usual variable environment, to interpret any assertion. This heap or more generally resource, had an algebraic structure to it: the most fundamental one being a partial commutative monoid (PCM), with the binary operator dictating when it is valid to combine to heaps. Many more complex algebras for resources were studied in later works [Calcagno et al. \[2007\]](#); [Pym et al. \[2004\]](#). Separation logic was identified as a natural tool to reason about imperative programs mutating heap based memory at a suitable level of abstraction. It then found success also in reasoning about concurrent programs [Brookes and O'Hearn \[2016\]](#).

2.3.1 Classical separation logic

The first separation logic was proposed by [O'Hearn et al. \[2001\]](#) for reasoning about programs which manipulate a heap. Before we move to probabilistic separation logic which is the main topic of the report, we first present separation logic in a simpler setting. Intuitively, separation logic is just first order logic with the additional connectives of the separating conjunction and separating implication (notation: $*$ and $\multimap$). These separating connectives are defined with respect to a *resource*. In the original separation logic we have a simple resource which is a finite partial function from some a set denoted Locations to a set Values (fig ??). The notation $h_0 * h_1$ is a partial binary operation denoting the union of h_0 and h_1 if they are disjoint. So we see that in order for a separating conjunct to hold we need to split the resource appropriately such that each part satisfies a conjunct. This is the principle behind all separation logics. Each proposition “owns” unique resources. One can imagine how each proposition makes assertions about parts of a program, and it is made explicit which parts of the heap, that part of the program “owns” and is thus able to modify. In short the logic allows reasoning in a resource-compositional manner. And specifically classical separation logic reasons about “memory” in languages like C, where the programmer must consciously manage the memory.

The other two *spatial connectives* are emp and $\multimap$. emp denotes the heap is an empty mapping, acting as the identity element to a monoid formed with $*$. $\multimap$ is to $*$ what $\rightarrow$ is to $\wedge$. Intuitively, $P \multimap Q$ holds under a heap q if P holds under some heap p for which the combined resource $q = p * q$, satisfies the goal Q .

Next observe the definitions of *forall quantification*. We quantify over the set Values that the given variable used to define P can take. Though we haven't introduced the programming language that this logic reasons about, note that the variables form the bridge between the language and logic, since these variables are shared between programs and assertions we make about the program. We do not go into the specifics of the introducing the programming language for this particular logic, since that takes us too far afield.

2.3.2 Zoo of separation logics

In this section we see increasingly abstract presentations of separation logic to spot patterns amongst them, and give definitions of characteristics that help classify separation logics. Identifying these characteristics and “telling” Iris about it, lets us effectively use the proof automation/engineering developed by the Iris framework for our probabilistic separation logic (which will be presented later).

Abstract resources

It was hinted in the previous section emp and $*$ is like a monoid. In general, what we mean by a “resource” can be modelled with some algebraic structure. The spatial connectives $*$ and $\multimap$ are both defined using the binary operation $\star$. At the core of a separation logic is the partial commutative monoid (PCM) formed by emp and $\star$. A partial monoid is just a monoid accounting to the fact that $\star$ is a partial operation. So we guard all the laws with existence checks. This turns out to be quite messy in a formalism, as seen for example in the `assoc` law. The below code shows the formalisation used taken from [Bppl.Lilac.Krm](#).

```
notation a:arg " * " b:arg => PcmBase.binop a b

class Pcm (α : Type*) extends One α where
  binop : α → α → Option α -- partial binary operation denoted `*`
  one_mul : ∀ a : α, 1 * a = a
```

Heaps $\triangleq$ Locations $\rightarrow_{\text{fin}}$ Values Stores $\triangleq$ Variables $\rightarrow_{\text{fin}}$ Values

(a) Basic sets and structures

B	$::=$	$True \mid False$	
E, F	$::=$	$\mathbb{N}$	
P, Q, R	$::=$	$B \mid E \mapsto F$	Atomic Formulae
		$\mid \text{false} \mid P \Rightarrow Q \mid \forall x.P$	Classical Logic
		$\mid \text{emp} \mid P * Q \mid P \multimap Q$	Spatial Connectives

(b) Syntax of separation logic

s, h	$\models$	B	iff	B
s, h	$\models$	$E \mapsto F$	iff	$\llbracket E \rrbracket_s \in \text{dom}(h)$ and $h(\llbracket E \rrbracket_s) = \llbracket F \rrbracket_s$
s, h	$\models$	false	never	
s, h	$\models$	$P \Rightarrow Q$	iff if	$s, h \models P$ then $s, h \models Q$
s, h	$\models$	$\forall x.P$	iff $\forall v \in \text{Values}$.	$[s \mid x \mapsto v], h \models P$
s, h	$\models$	emp	iff	$h = []$ is the empty heap
s, h	$\models$	$P * Q$	iff $\exists h_0, h_1$.	$h_0 \star h_1 = h$, $s, h_0 \models P$ and $s, h_1 \models Q$
s, h	$\models$	$P \multimap Q$	iff $\forall h'$.	if $h' \star h$ exists and $s, h' \models P$ then $s, h \star h' \models Q$

(c) Semantic interpretation

Figure 2.7: Definitions of separation logic

```

comm : ∀ a b, a * b = b * a
assoc : ∀ a b c : α,
  ((*) <$> (a * b) <*> (some c)).join = ((* <$> (some a) <*> (b * c)).join

class Krm (α : Type*) extends Pcm α, Preorder α where
  le_mul_mono : ∀ x x' y y' p' : α,
    x ≤ x' → y ≤ y' →
      (x' * y' = some p') → ∃ p, (x * y = some p) ∧ p ≤ p'

```

Lean follows a haskell like syntax. Above we define the typeclass representing PCM and KRM. Pcm extends One, which gives a $(1 : \alpha)$ which we interpret as emp. assoc is using functor syntax (from haskell) and `join : Option (Option α) → Option α`. This code is included to define precisely what is meant by partial monoid and krm, and to familiarise the reader with lean syntax.

The more interesting development is the order structure in the algebra. This can be motivated by wanting to model an affine separation logic (defined in the next section). Intuitively an affine separation logic always allows a larger heap to validate an assertion if a smaller heap does so too. This was not the case for the classical separation logic presented in the previous section which is a linear separation logic. Concretely only the single celled heap $\{1 \mapsto 42\}$ would validate the assertion $1 \mapsto 42$, and importantly $\{1 \mapsto 42, 2 \mapsto 42\}$ would not validate the same assertion. However, in an affine separation logic, any heap which contains $\{1 \mapsto 42\}$ would validate the assertion. Each of these flavours of separation logic are useful in different contexts. Classical separation logic is useful in a language with manual memory management while affine separation logic is used in a language with garbage collection. We see that we want an ordering in our algebra to express this “contains” relation or more generally how “informative” a resource is. So we introduce a preorder to the algebra, turning the PCM into a Kripke resource monoid (KRM) [Galmiche et al., 2005]. Note that KRMs also encompass PCMs/linear case, if we simply set the preorder to equality. For the affine heap-based separation logic, the preorder is subset inclusion on the domain of heaps. The standard connectives of separation logic (those connectives shared by all separation logics) can be generated from a KRM, which is the approach taken in the Lilac formalisation.

Logic of Bunched Implications

Abstracting even further, we would like to get rid of explicit mentions of the resource and the satisfiability relation $(s, h \models P)$ and talk instead about entailments of the form $\vdash P$ or $P \vdash Q$. The *logic of bunched implications* (BI) as presented by O’Hearn and Pym [1999] was what classical separation logic was based on. It can be described as a logic in which the familiar “intuitionistic” connectives of first order logic are combined with emp, * and $\multimap$ (called spatial connectives, alluding to heaps), in the same logic. We can recover the notion of resources by our interpretation of logical entailment: $P \vdash Q$ is defined as (using connectives in the meta-logic) $\forall s, h (s, h \models P \Rightarrow s, h \models Q)$. The definition of $\vdash$ is in general specific to each logic, but in practice (for separation logic) usually has the above shape. Having established this greater level of abstraction, we can more succinctly define some characteristics of different separation logics which will come in handy when instantiating the Iris framework. All subsequent figures in this section are reproduced from [Krebbers et al., 2018].

First let’s precisely define the relation between * and $\multimap$ (identical to $\wedge$ and $\rightarrow$)

The rest of this section reproduces some figures from [Krebbers et al., 2018].

$$\begin{array}{c}
 \text{emp} * P \dashv\vdash P \\
 P * Q \vdash Q * P \\
 (P * Q) * R \vdash P * (Q * R)
 \end{array}
 \qquad
 \begin{array}{c}
 \text{*}-\text{MONO} \\
 \frac{P_1 \vdash Q_1 \quad P_2 \vdash Q_2}{P_1 * P_2 \vdash Q_1 * Q_2}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{*}-\text{INTRO} \\
 \frac{P * Q \vdash R}{P \vdash Q \multimap R}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{*}-\text{ELIM} \\
 \frac{P \vdash Q \multimap R \quad P * Q \vdash R}{P \vdash R}
 \end{array}$$

<insert: If I add the one_le, does the KRM become an affine logic generator?>

Next we define the affine propositions: P as any proposition satisfying for all Q $P * Q \vdash Q$. In other words P can always be dropped. An affine separation logic is one where all propositions are affine. Affine propositions can also be equivalently defined as $P * \text{True} \dashv\vdash P$ or $\text{True} \dashv\vdash \text{emp}$. Another class of propositions is persistent propositions. Those propositions which may be duplicated to establish the goal. So for instance if P is persistent then $P \vdash P * P$. <insert: Why is this

not the definition?>. Intuitionistic propositions are a final class of propositions which are both affine and persistent. Working with intuitionistic propositions in the premise feels like working with the familiar first order (intuitionistic) logic, since we don't at all need to keep track of how many times these propositions are used. Because of this Iris treats intuitionistic propositions specially, keep them separate from the rest of the “spatial” propositions in the premise. So a user of the iris proof-mode will see an “intuitionistic context”, following rules familiar to them from using the host theorem prover (Rocq or Lean), and additionally the spatial context, which they will need to treat more carefully. Examples follow in the next section.

2.3.3 Iris and MoSeL

Iris is a general framework for tactic based proofs in a separation logic. Iris (specifically MoSeL) can be understood according to the layers of abstraction it provides.

```
KRM -> BI                                ->                                Iris Proof Mode
|_ Probability Specific connectives -> typeclass instances -|
```

This is sort of the general structure of how Iris is meant to be used by someone instantiating it. You start with a KRM or similar algebraic structure which is the resource model, from which there is pre-determined construction of a BI (those connectives common to all separation logic). Given this Iris proof mode will be aware of the standard properties of a separation logic. For example, it is able to prove the following goal, note it has implicitly applied both commutativity separating conjunct <insert: example 1> We are also able to introduce both variables and premises with the same command, to build a context of everything we have to prove our goal. <insert: example 2> What is unique about the Iris proof mode when compared to Lean or Rocq, is that it has an additional spatial context. <insert: example with spatial context> In short it is engineered to provide an interface which mimics the natural reasoning style of user. The IPM understands separation logic and endeavours to automate common boring tasks.

Though these seem like rather basic conveniences they are quite difficult to provide in a manner extensible to all separation logics. In order to understand how the proof mode works (as we will be instantiating Iris), it is first necessary to understand tactic based proofs in general.

2.4 Theorem Proving

2.4.1 Dependent type theory

Examples in this section are based on the excellent introduction to the calculus of inductive constructions by [Paulin-Mohring \[2014\]](#).

Foundational theorem proving is proving theorems with only the axioms (foundations) of mathematics assumed. Likewise foundational software verification for proving theorems (properties) of programs.

Here we focus on the theory of foundational theorem proving and in the next (2.4.2) we focus on how these ideas scale to large projects in practice. Note that we are interested in “proving theorems” as software verification is just a particular instance of theorem proving (using program logics, which is covered in detail in the rest of this report).

We now focus on a specific “Foundation of Mathematics” which has proved ideal for theorem proving in practice, as displayed by the most successful theorem provers - Rocq, Lean, Agda, Idris - all being based on dependent type theory. Dependent type theory has the remarkable property of encoding both propositions (synonymous to theorems) and proofs on one side and types and programs on the other, as part of the same language. These two sides are in-fact formally indistinguishable. The Curry-Howard isomorphism (or propositions as types method) is an observation that the rules governing logic (introduction elimination rules), are much the same as the inference rules governing typing judgements. So the two can be used interchangeably. Certain types in our type theory will be interpreted as propositions, and terms of that type will be proofs of that proposition!

Below is a description of Calculus of Constructions (CoC), a flavour of dependent type theory on which Rocq (CoC and its successors were developed for Rocq) and Lean are based. Agda is based on Martin Lof Type Theory (MLTT) which differs in a matter of predicativity. We will explain and revisit this briefly at the end.

Pure CoC consists of very few rules and constructs and it is very surprising that it is a valid foundation for all mathematics. We start with a typed lambda calculus in which both the terms and the types are given by the *same* syntax. This is known as a pure type system (PTS). Instead of function/arrow type we have the dependent product, $\Pi(x : A), B(x)$, where B may be defined using x . Additionally, (as there is no separate syntax for types) all types have *sort*. In other words, every construction in the calculus must have a type, which itself will have a type and so on, infinitely. A has type **Type**₁ (or *sort* **Type**₁, to acknowledge that A is itself a type). There is actually an infinite hierarchy of these types: $A : \mathbf{Type}_1 : \mathbf{Type}_2 : \mathbf{Type}_3 \dots$, where the subscripts are indices known as *universes*.

The type of identity functions A is given by $\Pi(_ : A), A$. We use the notation $A \rightarrow A$ in this case since it is not dependent. For completeness the identity function is typed $(\lambda x : A \mapsto x) : \Pi(_ : A), A$. As a first example of a dependent type take the type of the polymorphic identity function, which takes a type and gives the identity function for that type: $\Pi(A : \mathbf{Type}_i), A \rightarrow A$. So $A \rightarrow A$ (or $\Pi(_ : A), A$) is using A in its definition therefore dependent. Again for completeness we type the term which is the polymorphic identity function: $\lambda A : \mathbf{Type} \mapsto \lambda x : A \mapsto x : \Pi(A : \mathbf{Type}_i), A \rightarrow A$. We will not go into the detail of universe polymorphism which strictly speaking should be considered since the index i is unquantified in the above expressions.

We now move to expressing logic in CoC, but before doing so we introduce the sort **Prop** : **Type**₁, so that the (infinite) set of sorts is given by $\mathcal{S} := \{\mathbf{Prop}\} \cup \bigcup_{i \in \mathbb{N}} \{\mathbf{Type}_i\}$. **Prop** is special and is *impredicative* as we will see in 2.4.1. We map first order logic propositions to types of sort **Prop**. For example, if $A : \mathbf{Prop}$ and $B : \mathbf{Prop}$, then logical implication $A \Rightarrow B$ corresponds to the type $A \rightarrow B$. This is justified by the meaning of implication to be being able to prove B given a proof of A . Forall quantification (also) corresponds to dependent product types: given some $T : \mathbf{Type}_i$ and $P : \mathbf{Prop}$, the logical statement $\forall x : T, P(x)$ corresponds to the type $\Pi(x : T), P(x)$. This also make sense since to prove a universally quantified statement we need a proof of $P(x)$ for every x , which is what a term of type $\Pi(x : T), P(x)$ would give. We will use the notation $\forall x : T, P$ for $\Pi(x : T), P(x)$, freely, as is standard in Lean.

Dependent type theory as a foundation is an example of *constructive mathematics* because proofs are always built by assembling the required data as seen in above example. With that intuition in mind lets show how some other logical connectives are represented:

As seen in this example, the standard recipe in constructive mathematics is to specify a proposition by what is required prove it. In the next example we see representation also arises from what the connective enables us to prove. Take $\text{False} := \forall (P : \mathbf{Prop}). P : \mathbf{Prop}$. First notice that if we have a proof of False , that is a term of type $\forall (P : \mathbf{Prop}). P$, we would get the proof of every any proposition P , out of that, encoding false entails anything. Also, it is not possible to construct a term of type $\forall (P : \mathbf{Prop}). P$, ie, there is no proof of False . A strange thing that one might notice is that we are quantifying over all everything in $\mathbf{Prop}$, in order to obtain something of type $\mathbf{Prop}$.

Predicativity

In this section we continue with more examples of logic using dependent type theory since it comes up in a different context in the definition of Iris, which took a while to figure out.

Let's now introduce all the typing rules. Remember $\mathcal{S} := \{\mathbf{Prop}\} \cup \bigcup_{i \in \mathbb{N}} \{\mathbf{Type}_i\}$.

We notice that there are two typing rules for the dependent product: **PI-PROP** for sort **Prop** and **PI-TYPE** for all other sorts. **PI-TYPE** is saying that the dependent product falls into a sort which is the max of the sort of its domain and codomain. While there is also a special case rule **PI-PROP**, which says that **PI-PROP** says that regardless of the domain, if the codomain is in **Prop**, then the Π -type stays in **Prop**. We say that the sort **Prop** is impredicative, because elements in it are defined by quantifying over **Prop** itself! Remembering $\text{False} := \Pi P : \mathbf{Prop}, P : \mathbf{Prop}$, we see exactly this. And suppose that we instead were to type this with the rule **PI-TYPE** and $\mathbf{Prop} \triangleq \mathbf{Type}_0$, then the domain, $\mathbf{Prop} : \mathbf{Type}_1$ and codomain $P : \mathbf{Prop}$ would give the Π -type sort $\mathbf{Type}_1$, making **Prop**

$$\begin{array}{c}
\frac{\Gamma \vdash}{\Gamma \vdash \text{Prop} : \text{Type}_1} \text{TYPE-PROP} \qquad \frac{\Gamma \vdash}{\Gamma \vdash \text{Type}_i : \text{Type}_{i+1}} \text{TYPE-FORM} \\
\\
\frac{\Gamma \vdash x : A \in \Gamma}{\Gamma \vdash x : A} \text{VAR} \qquad \frac{\Gamma \vdash A : s \quad x \notin \Gamma \quad s \in S}{\Gamma, x : A \vdash} \text{CTX-EXT} \\
\\
\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \text{fun } x : A \Rightarrow t : \Pi x : A, B} \text{FUN-INTRO} \qquad \frac{\Gamma, x : A \vdash B : \text{Prop}}{\Gamma \vdash \Pi x : A, B : \text{Prop}} \text{PI-PROP} \\
\\
\frac{\Gamma \vdash A : \text{Type}_i \quad \Gamma, x : A \vdash B : \text{Type}_j}{\Gamma \vdash \Pi x : A, B : \text{Type}_{\max(i,j)}} \text{PI-TYPE} \qquad \frac{\Gamma \vdash f : \Pi x : A, B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B[a/x]} \text{APP} \\
\\
\frac{\Gamma \vdash t : A \quad \Gamma \vdash B : s \quad A \preceq B}{\Gamma \vdash t : B} \text{CONV}
\end{array}$$

Figure 2.8: Inference rules for the purely functional part of CIC [Paulin-Mohring, 2014]

predicative. However, we would like $\text{False} : \text{Prop}$, so the impredicativity of Prop is desirable. Let's also recall the definition of forall quantification which also depended on predicativity: $\forall x : T, P$, we can quantify over values in much larger universes or quantify over all Props and still end up with a type in Prop . Impredicativity is powerful but fragile. It can be shown that if Type_1 is also impredicative, then this type theory would become inconsistent [Coquand, 1999].

As a final example let's have a look at $A \wedge B \triangleq (P : \text{Prop}). (A \rightarrow B \rightarrow P) \rightarrow P : \text{Prop}$. Or making things a little more explicit: $\wedge := (\lambda AB : \text{Prop}. (P : \text{Prop}). (A \rightarrow B \rightarrow P) \rightarrow P : \text{Prop} \rightarrow \text{Prop} \rightarrow \text{Prop})$, where we are first using lambda abstractions and then a Π -type construction.

Proof Relevance

Finally, Proof Relevance separates the type theories of Lean and Rocq. Lean is Proof irrelevant meaning terms of types of Prop , are definitionally equal. So for instance if we have a structure bundling data and a proof of a property like so in Lean:

```

structure MeasurableFun (α β : Type*) [MeasurableSpace α] [MeasurableSpace β] where
  /-- underlying function -/
  toFun : α → β
  meas : Measurable toFun

```

the equivalence of two `MeasurableFuns` is determined solely by the equality of the `toFun`, since `meas` is a (dependent) proof and will be definitionally equal if `toFun` is equal. In practice this is incredibly convenient, and is applied automatically throughout the codebase to simplify proof objectives. It was always used in an elegant abstraction in establishing that certain constructions form a KRM, which we will detail later.

The choices made in lean's type theory to bring about proof irrelevance does have some major theoretical drawbacks such as leading to undecidability of definitional equality [Carneiro, 2019, see chapter 3], which however don't seem to matter too much in practice.

Calculus of Inductive Constructions

Mention here that 1) Inductive families... 2) strict positivity

In this section we hope to have gotten across that theorem proving in a dependent type theory is exactly type checking of terms constructed by the user. This is all that is needed to motivate the next section on how this is partially automated and scaled in Rocq and Lean (going into more Lean specific detail). We also introduced the notions of predicativity and proof relevance, both of which I encountered over the course of the project without have previously heard of it. Interestingly they also cleanly separate 3 of the most significant proof assistants. They will be mentioned briefly where relevant but is not particularly important in understanding most of this report.

2.4.2 Metaprogramming

We now know that constructing a proof involves assembling many subproofs according to the typing rules to obtain a term of a desired Type (actually Prop). It is however extremely time-consuming and error-prone to go down to this level of detail when constructing a proof. Pen and paper proofs operate at a much higher level, and the use of metaprogramming in Proof assistants, is motivated by the desire to remove the mental overhead of mechanised proofs, move closer to the style of pen and paper proofs.

“Tactics” are metaprograms meant for constructing proof terms. A Tactic block is started in Lean using *by* syntax: in our code if we wanted to a term (proof) of type $A : Prop$, we can write $(by \dots) : A$ (though the type A , would usually be inferred by lean). So how does the *by* tactic make it easier to construct the term? It provides a proof context, of premises and goals, which is actually additional state wrapped in a monad called `TacticM`. Inside a tactic block a sequence of `TacticM` monads are composed, each monad constructed using one Lean’s ecosystem of tactics, incrementally modifying the proof state until “all goals are solved”. And at the end of a tactic block, using the information acquired in the monadic state a term of the desired type should be constructed using the information collected in the `TacticM` monad.

We have been pretty vague about what this “proof context” and “goals” actually are so far. In lean there are a few more constructors for terms than in the simply types lambda calculus, on top of lambda abstraction and application. Terms are called `Expr` in lean. We introduce the constructor `mvar : MVarID → Expr`. For building an expression given a metavariable. This is at the heart of how tactics work. Tactics communicate to each other of what has been proven and what is left to prove using metavariables. We will illustrate this through a simple example reproduced from [leanprover community, 2024, chapter 4]. In the proof of the following theorem we show the evolution of the proof context, line by line.

```
example {α} (a : α) (f : α → α) (h : ∀ a, f a = a) : f (f a) = a := by
  apply Eq.trans
  · apply h
  · apply h
```

Metavariables are used to represent unknown terms of known types, which may be holes in larger terms or a goal we want to prove (term of a certain Prop that we are looking to construct). As soon as the tactic block starts with *by*, a metavariable `?m1` which is unassigned but has type $f (f a) = a$. Now we look at the state after the first `apply`. The types of `Eq.trans` and `?m1` can be unified, so the tactic succeeds and we have new proof state as follows:

To a user it would look like this:

```
...
⊢ f (f a) = ?b

...
⊢ ?b = a

...
⊢ a
```

But its helpful to see the state of metavariables explicitly:

```
?m1 := Eq.trans α (f (f a)) ?b a ?m2 ?m3
?m2 : f (f a) = ?b
?m3 : ?b = a
?b
```

where `Eq.trans : ∀ (α : Sort u), ∀ (a b c : α), a = b → b = c → a = c`.

The old goal `?m1` has been assigned (albeit with holes in the assignment). Two new goals `?m2` and `?m3` were created and an additional mvar `?b`, which is not a Prop was created as well. After the second `apply`, which acts on the goal `?m2`, we assign `?m2` and also assign `?b` through the unification process.

```
?m1 := Eq.trans α (f (f a)) ?b a ?m2 ?m3
?m2 := h (f a)
?m3 : ?b = a
?b   := f a
```

Only `?m3` needs to be assigned which is taken care of the final `apply`. At the end of the tactic block, we obtain the term `Eq.trans α (f (f a)) (f a) a (h (f a)) (h a)` which is typed as `f (f a) = a`, as desired!

2.5 Instantiating Iris

Theorem proving, can be split into two activities: writing definitions and proving theorems. The former doesn't seem difficult or time-consuming. It sounds reasonably easy to come up with a skeleton for the codebase, based on a paper. After a while working on this project I have realised this is not true, and indeed this is well known by both the theorem proving community, also true for professional mathematics and software engineering. Let's elaborate on the latter. Writing computer code is parallel to writing definitions (of data and algorithms). So definitions are indeed important and time-consuming in Lean as well. There is no parallel to writing theorems though, programs just need to convince themselves of the correctness.

A milestone was reached when the definitions of the language and logic were written down, with a set of abstractions to convince myself this development was sound. These definitions were composed of a separate syntax and semantics for both the language and the logic, using dependent de bruijn indices for both. These definitions would have sufficed to prove formally the correctness of all the proofs in the paper. However, of the project is not only to validate the soundness of the logic, but also to create a usable tool for proving properties of actual probabilistic programs ergonomically (using tactics). The best way to do so is to instantiate the Iris framework. And the interface provided by Iris was not compatible with current definitions. Below we explain why it wasn't compatible and how Iris' interface is motivated.

2.5.1 Deep vs Shallow Embeddings

When defining a language or logic, within a meta-language (lean in our case), we are presented with 2 choices. A deep embedding; inductively define the syntax, and the denotational semantics becomes an inductive definition over the syntax. For clarity, here is an excerpt from a deep embedding of the Lilac logic:

<insert: complete this (code from c3ca63f)>

```
inductive Assertion : List Ty → List Ty → Type
| top : Assertion ds rs -- ds: deterministic context, rs: random variable context
| and : Assertion ds rs → Assertion ds rs → Assertion ds rs
| sep : Assertion ds rs → Assertion ds rs → Assertion ds rs
| forall (d : Ty) : Assertion (d :: ds) rs → Assertion ds rs
| forall_rv (r : Ty) : Assertion ds (r :: rs) → Assertion ds rs
| exists_rv (r : TyRand) : Assertion ds (r :: rs) → Assertion ds rs
...

def Assertion.denote {ds : List TyDet} {rs : List TyRand}
(P : Assertion ds rs) (γ : HList (⟦·⟧] ds) (D : RV (List.TProd (⟦·⟧] rs))) (ψ : PSp HC) : Prop :=
match P with
| top => True
| and P Q => P.denote γ D ψ ∧ Q.denote γ D ψ
| sep P Q => ∃ ψ1 ψ2 : PSp HC, ψ1 * ψ2 ≤ some ψ ∧ P.denote γ D ψ1 ∧ Q.denote γ D ψ2
| forall d P => ∀ x : ⟦d⟧, P.denote (x :: γ) D ψ
| forall_rv r P => ∀ X : RV ⟦r⟧, P.denote γ (X ; D) ψ
| exists_rv r P => ∃ X : RV ⟦r⟧, P.denote γ (X ; D) ψ
```

Figure 2.9: Excerpt of Deep Embedding of Lilac

On the other hand a shallow embedding where we fuse inductive structure of the syntax and the semantic denotation function together to simply. The denotations of terms in our object language get expressed directly in the meta language, and we lose the ability to distinguish entities in the object language from the meta-language. The previous point, in concrete terms is that we lose the ability to do induction over the object language now. Here is an excerpt of Lilac as a shallow embedding:

The shallow embedding had a much smaller surface and generally allowed the logic to piggyback on the forall and exists quantifiers of the meta language.

2.5.2 BI in Iris

Remember that in 2.3.2 we explained that BI (logic of bunched implications) is just abstraction which defining the standard connectives shared by every separation logic. Here is the definition of BI as a typeclass in [iris-lean](#) (a dependency of this project).

```
--
Require the definitions of the separation logic connectives and units on a carrier type `PROP`.
-/
class BIBase (PROP : Type u) where
  Entails : PROP → PROP → Prop
  emp : PROP
  pure : Prop → PROP
  and : PROP → PROP → PROP
  or : PROP → PROP → PROP
  imp : PROP → PROP → PROP          -- implies
  sForall : (PROP → Prop) → PROP
  sExists : (PROP → Prop) → PROP
  sep : PROP → PROP → PROP          -- separating conjunct
  wand : PROP → PROP → PROP         -- magic wand
  persistently : PROP → PROP
  later : PROP → PROP
```

Firstly **Prop** is the universe of propositions, part of Lean’s syntax. While PROP denotes the proposition in the object logic (Lilac in our case). The typeclass is called BIBase since there is another typeclass BI extending BIBase which states the axioms of BI. As explained in 2.3.2, the instantiation of Entails can interpret a BI as a separation logic. Entails says what it means for a PROP to be true by translating into the meta language’s **Prop**. persistently and later is omitted since the former is an orthogonal concern and the latter is irrelevant. emp can be understood as the unit of the monoid with which we will instantiate BI. There are several binary operators. The most important connectives are sForall and sExists. The original definition from iris-roq is more straightforward:

```
iForall : ∀ A : Type u', (A → PROP) → PROP
iExists : ∀ A : Type u', (A → PROP) → PROP
```

We quantify over the type A of variables that we want to quantify over, since iris can’t know what type a variables a specific logic may want to quantify over, and indeed as in Lilac there may be multiple types (Lilac has two of each quantifier). We have named the above quantifier iForall, with i short for indexed, since when quantifying over types we refer to the types (like A) as *type indices*. iForall and iExists is impredicative (see 2.4.1), meaning we quantify in universes larger than that of the type being constructed. u' is universe level, a natural number. Indeed, impredicativity is required here to provide complete flexibility in what variables an instantiating logic can then quantify over.

Now lets look at sForall and sExists, with s short for *set*. Note that in theorem provers, Sets (say Set A) are represented as $A \rightarrow \mathbf{Prop}$. So the signature for the quantifiers says that given a set of PROP we construct a PROP. The reason for this less intuitive version of quantification is that iForall (iExists) can be defined in terms of sForall (sExists). And the quantified types are not restricted to a predetermined universe u'.

2.5.3 First Attempt: Deeply embedded logic instantiation

There is a result in Lilac [Li et al., 2023, see Appendix B.10] called the substitution lemma which uses induction over the syntax of Lilac propositions. Supposing that induction over the syntax of the logic is important/necessary, a next step is to try to instantiate BIBase above with the deep embedding from Fig 2.9. We may try something like so:

```
instance (ds rs : List Ty) : BIBase (Assertion ds rs) where
  Entails P Q := ∀ γ D ψ, P.denote γ D ψ → Q.denote γ D ψ
  and       := Assertion.and
  ...
```

Where we simply instantiate each of the constructors with syntax, and called the denotation function in `Entails`. This works nicely until we get to the instantiation of the quantifiers. We now need some syntactic version of `sForall` and not just the `forall`/`forall_rv` from Fig 2.9. Though the latter is all we need for our logic instantiating iris demands the flexibility to quantify over any type, not just the object language types. Let's introduce the following syntax into our language:

```
inductive Assertion' : List TyDet → List TyRand → Type
  | sForall : (Assertion' ds rs → Prop) → Assertion' ds rs
  ...
```

This looks like a valid lean definition, but it is not, since it fails a syntactic requirement of inductive types called *strict positivity*. The type being defined inductively may only appear in *positive* locations, which means they are not allowed to occur in the input of an arrow type. Even if we suppose the `iForall` definition was used in BI, we still end up with nuanced problems with mixing semantics into the syntax because of the quantifiers. We spare further details, but the conclusion is that the impredicativity of quantifiers in BI disallows usage of a deep embedding of the logic.

2.5.4 Shallow embedding of the logic

2.6 Deparametrisation of LProp

A major concern throughout the mechanisation was the definition of `LProp`, and how amenable it might be to variable substitution. Remember that we were forced to forego the convenience of an induction principle over the lilac connectives (or constructors in this context). This was indeed due to the generality of iris without which we wouldn't be able to use it in the first place. This seemingly insignificant issue tended to have a significant effect on the decisions made. We explain below, why the notion of substitution as defined in the paper, is incompatible with iris.

Lilac has two contexts, the random and deterministic contexts. The latter is only required when the logic supports reasoning about conditioning. Remember that conditioning takes random variable and fixes its values, i.e. makes it deterministic. And only for that deterministic value do we need the deterministic context and otherwise, every variable in our language corresponds to a random variable. So we will focus on the random contexts. Take the quantifier $\forall_{rv}$:

$$\gamma, D, \mathcal{P} \models \forall_{rv} X : A. P \quad \text{iff} \quad \text{for all } X : RVA(\gamma, (D, X), \mathcal{P}) \models P$$

We are quantifying over RVA , which means variables in Lilac are defined to have this type. Remember RVA means $HC \xrightarrow{m} A$ (with additional finite footprint property not relevant here). Substitution is the process of transforming from one LProp to another by specifying a mapping from one variable to another. However, definition of substitution given in the paper looks like this:

$$\begin{aligned} S &\in \llbracket \Delta' \rrbracket \rightarrow \llbracket \Delta \rrbracket \\ \top[S] &= \top \\ \perp[S] &= \perp \\ (P \wedge Q)[S] &= (P[S] \wedge Q[S]) \\ (P \vee Q)[S] &= (P[S] \vee Q[S]) \\ (P \rightarrow Q)[S] &= (P[S] \rightarrow Q[S]) \\ (P * Q)[S] &= (P[S] * Q[S]) \\ (P - * Q)[S] &= (P[S] - * Q[S]) \\ (\forall_{rv} X : A. P)[S] &= \forall_{rv} X : A. P[S \times 1_A] \\ (\exists_{rv} X : A. P)[S] &= \exists_{rv} X : A. P[S \times 1_A] \\ &\dots \end{aligned}$$

It is helpful to have the following mental picture of the meaning of an LProp as a function $HC \xrightarrow{m} \llbracket \Delta \rrbracket \xrightarrow{m} A$, pick an element ω of HC (which is like sampling a random number). Once we do this

the randomness is gone and the maps forward are deterministic. ω maps to values of variables, $\llbracket \Delta \rrbracket$, as an intermediate step. Then these variables deterministically map to some expression of type A . With this in mind note that there's a mismatch in what we would like substitution to be, replacing $X : HC \xrightarrow{m} A$ with $Y : HC \xrightarrow{m} A$ which is used in an $LProp$, with the above definition of substitution, which only starts at the intermediate $\llbracket \Delta \rrbracket$ step.

At a high level the Iris proof mode (indeed any proof mode) works by splitting an $LProp$ into a context of premises and variables, and a goal. The variables are of type $RV A$ and if the substitutions are not with $RV A$ as domain, the types just won't line up.

In order to explain further we need an example of where substitution needs to be used. One canonical usage of substitution is in the proof rule wp_ret :

$$Q[\llbracket M \rrbracket / X] \vdash wp(\llbracket retM \rrbracket, X.Q)$$

saying that the random variable obtained by sampling the program $\llbracket retM \rrbracket$ is just always the denotation of M , $\llbracket M \rrbracket$. $\llbracket M \rrbracket$ substitutes the variables X which is free in the $LProp$ Q , so substitution is essentially removal of free variables. Initial attempts at defining wp_ret tried to adapt the proposed at the $\llbracket \Delta \rrbracket$ level, but this led to a painfully convoluted system before finally hitting a dead end with this approach. We try to outline the old design with the old definitions of $LProp$. At a high level, the above notion of substitution can be made to work

The development got quite far with the $LProp$ being parametrised with the type of the deterministic context. However, that started causing serious issues when stating the proof rules, because of the book-keeping that needs to be done with always keeping track of the number of variables in context. Not to mention this bookkeeping is quite difficult since we always need to keep proving that the the contexts are a measurableSpace.

2.6.1 Substitution inconsistency

Lilac

Bibliography

- Bob Carpenter, Andrew Gelman, Matthew D. Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1):1–32, 2017. doi: 10.18637/jss.v076.i01. URL <https://www.jstatsoft.org/index.php/jss/article/view/v076i01>.
- Praveen Narayanan, Jacques Carette, Wren Romano, Chung-chieh Shan, and Robert Zinkov. Probabilistic inference by program transformation in hakaru (system description). In *International Symposium on Functional and Logic Programming - 13th International Symposium, FLOPS 2016, Kochi, Japan, March 4-6, 2016, Proceedings*, pages 62–79. Springer, 2016. doi: 10.1007/978-3-319-29604-3_5. URL http://dx.doi.org/10.1007/978-3-319-29604-3_5.
- Frank Wood, Jan Willem van de Meent, and Vikash Mansinghka. A new approach to probabilistic programming inference. In *Proceedings of the 17th International conference on Artificial Intelligence and Statistics*, pages 1024–1032, 2014.
- Sam Staton. Commutative semantics for probabilistic programming. In *Programming Languages and Systems: 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22–29, 2017, Proceedings*, page 855–879, Berlin, Heidelberg, 2017. Springer-Verlag. ISBN 978-3-662-54433-4. doi: 10.1007/978-3-662-54434-1_32. URL https://doi.org/10.1007/978-3-662-54434-1_32.
- Jialu Bao, Emanuele D’Osualdo, and Azadeh Farzan. Bluebell: An alliance of relational lifting and independence for probabilistic reasoning. *Proc. ACM Program. Lang.*, 9(POPL), January 2025. doi: 10.1145/3704894. URL <https://doi.org/10.1145/3704894>.
- Janine Lohse, Tim Rohde, Jimmy Xin, Niklas Mück, Iona Kuhn, Derek Dreyer, Deepak Garg, and Emanuele D’Osualdo. First steps towards probabilistic iris: Harmonizing independence, conditioning, and dynamic heap allocation, 2026. URL <https://arxiv.org/abs/2605.13765>.
- Rémy Degenne. Markov kernels in mathlib’s probability library, 2025. URL <https://arxiv.org/abs/2510.04070>.
- Chris Heunen, Ohad Kammar, Sam Staton, and Hongseok Yang. A convenient category for higher-order probability theory. In *2017 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, page 1–12. IEEE, June 2017. doi: 10.1109/lics.2017.8005137. URL <http://dx.doi.org/10.1109/LICS.2017.8005137>.
- Michikazu Hirata, Yasuhiko Minamide, and Tetsuya Sato. Semantic Foundations of Higher-Order Probabilistic Programs in Isabelle/HOL. In Adam Naumowicz and René Thiemann, editors, *14th International Conference on Interactive Theorem Proving (ITP 2023)*, volume 268 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 18:1–18:18, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. ISBN 978-3-95977-284-6. doi: 10.4230/LIPIcs.ITP.2023.18. URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2023.18>.
- Reynald Affeldt, Cyril Cohen, and Ayumu Saito. Semantics of probabilistic programs using s-finite kernels in dependent type theory. *ACM Trans. Probab. Mach. Learn.*, 1(3), August 2025. doi: 10.1145/3732291. URL <https://doi.org/10.1145/3732291>.

- Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich. *Theorem Proving in Lean 4*. 2024. URL https://leanprover.github.io/theorem_proving_in_lean4/. with contributions from the Lean Community.
- Mario Carneiro. The type theory of lean. Master’s thesis, Carnegie Mellon University, 2019. URL <https://github.com/digama0/lean-type-theory/releases/download/v1.0/main.pdf>.
- Sheldon Axler. *Measures*, pages 13–72. Springer International Publishing, Cham, 2020. ISBN 978-3-030-33143-6. doi: 10.1007/978-3-030-33143-6_2. URL https://doi.org/10.1007/978-3-030-33143-6_2.
- John M. Li, Amal Ahmed, and Steven Holtzen. Lilac: A modal separation logic for conditional probability. *Proc. ACM Program. Lang.*, 7(PLDI), June 2023. doi: 10.1145/3591226. URL <https://doi.org/10.1145/3591226>.
- Michèle Giry. A categorical approach to probability theory. In B. Banaschewski, editor, *Categorical Aspects of Topology and Analysis*, pages 68–85, Berlin, Heidelberg, 1982. Springer Berlin Heidelberg. ISBN 978-3-540-39041-1.
- Peter O’Hearn, John Reynolds, and Hongseok Yang. Local reasoning about programs that alter data structures. In Laurent Fribourg, editor, *Computer Science Logic*, pages 1–19, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg. ISBN 978-3-540-44802-0.
- Cristiano Calcagno, Peter W. O’Hearn, and Hongseok Yang. Local action and abstract separation logic. In *22nd Annual IEEE Symposium on Logic in Computer Science (LICS 2007)*, pages 366–378, 2007. doi: 10.1109/LICS.2007.30.
- David J. Pym, Peter W. O’Hearn, and Hongseok Yang. Possible worlds and resources: the semantics of bi. *Theoretical Computer Science*, 315(1):257–305, 2004. ISSN 0304-3975. doi: <https://doi.org/10.1016/j.tcs.2003.11.020>. URL <https://www.sciencedirect.com/science/article/pii/S0304397503006248>. Mathematical Foundations of Programming Semantics.
- Stephen Brookes and Peter W. O’Hearn. Concurrent separation logic. *ACM SIGLOG News*, 3(3):47–65, August 2016. doi: 10.1145/2984450.2984457. URL <https://doi.org/10.1145/2984450.2984457>.
- D. Galmiche, D. Méry, and D. Pym. The semantics of bi and resource tableaux. *Mathematical Structures in Computer Science*, 15(6):1033–1088, 2005. doi: 10.1017/S0960129505004858.
- Peter W. O’Hearn and David J. Pym. The logic of bunched implications. *Bulletin of Symbolic Logic*, 5(2):215–244, 1999. doi: 10.2307/421090.
- Robbert Krebbers, Jacques-Henri Jourdan, Ralf Jung, Joseph Tassarotti, Jan-Oliver Kaiser, Amin Timany, Arthur Charguéraud, and Derek Dreyer. Mosel: a general, extensible modal framework for interactive proofs in separation logic. *Proc. ACM Program. Lang.*, 2(ICFP), July 2018. doi: 10.1145/3236772. URL <https://doi.org/10.1145/3236772>.
- Christine Paulin-Mohring. Introduction to the calculus of inductive constructions. 11 2014.
- Thierry Coquand. An analysis of girard’s paradox. 10 1999.
- leanprover community. Metaprogramming in lean 4. <https://leanprover-community.github.io/lean4-metaprogramming-book/>, 2024. Online book.